AWS IAM Lesson

Skills you will learn

Cloud Computing: Identity & Security

Below is a comprehensive, up-to-date introduction to **AWS Identity and Access Management (IAM)**. It mixes core concepts with hands-on steps so you can try everything yourself right away.

1. Why IAM exists

AWS is built around "secure by default." IAM is the service that lets you define **who** (principal) can do **what** (action) on **which** resources, **under which conditions**, without ever sharing long-lived passwords or keys. Every other AWS security feature (MFA, KMS encryption keys, WAF policies, etc.) ultimately plugs into IAM evaluation logic.

2. Key building blocks

Concept	What it is	Typical use
Root user	The account-owner identity created at sign-up.	Emergency tasks only; always MFA-protected.
	Long-term identity for a	Legacy scripts or when you cannot assume

Group	Container for users; policies attached to the group apply to every member.	"Developers," "Auditors," etc.
Role	Temporary identity that anyone/anything can *assume*. No static credentials.	EC2 instances, Lambda, CI/CD pipelines, cross-account access.

Concept	What it is	Typical use
Policy	JSON that lists **Effect**, **Action**, **Resource**, optional **Condition**.	Grants or restricts permissions.
Identity Center	Successor to AWS SSO; your workforce login and SSO hub.	Centralised access to many AWS accounts & SaaS apps. ([AWS Documentation][1])
IAM Roles Anywhere	Extends IAM roles to on-prem or multi-cloud servers using X.509 certs.	Hybrid workloads without long-term keys. ([Amazon Web Services][2])
IAM Access Analyzer	Automated reasoning engine that tells you *who* can reach a resource.	Detect unintended public or internal access. New 2025 "Internal Access Findings" shows org-wide access paths. ([Amazon Web Services][3])
Amazon Verified Permissions	Managed authZ service for your own apps, using the Cedar policy language.	Fine-grained, app-level RBAC/ABAC. ([Amazon Web Services][4])

3. How policy evaluation works (mental model)

1. **Explicit deny** wins.
2. If no explicit deny, an explicit **allow** wins.
3. If neither, request is implicitly denied.

AWS merges *all* applicable policies at evaluation time: identity-based, resource-based, permission boundaries, session policies, plus higher-level controls such as Service Control Policies (SCPs) in AWS Organizations.

4. Best-practice checklist (2025 edition)

- **Use roles with temporary credentials everywhere**—for humans via Identity Center and for workloads via IAM roles or Roles Anywhere. ([AWS Documentation][5])
- **MFA** for every interactive user—including the root user.
- **Least privilege**: start with zero, let Access Analyzer or the IAM Policy Generator suggest only what you need. ([Amazon Web Services][6])
- **Block public access** in S3 unless there is a verified business need.
- **Use permission boundaries & SCPs** to keep accounts and teams in their lane.

- **Rotate/disable old access keys**; adopt modern auth patterns (OIDC, STS) instead of static keys. ([Amazon Web Services][6])

5. Hands-on mini-lab

> ***Goal:*** *create a secure developer user, grant read-only S3 access, launch an EC2 instance that can list objects in a specific bucket, and verify everything with the AWS CLI.*

5.1 Prerequisites

- AWS account where you are an administrator.
- AWS CLI v2 installed locally.

5.2 Create a developer group & user

1. Console → **IAM** → **Groups** → **Create group** → name `dev-readonly`.
2. Attach policy **`AmazonS3ReadOnlyAccess`**.
3. **Users** → **Create user** → name `alice`, add to `dev-readonly`, *disable* access keys, enable **AWS IAM Identity Center** credentials instead (recommended) **or** create an access key for CLI testing only.
4. Require MFA on first sign-in.

5.3 Create an S3-access role for EC2

1. **Roles** → **Create role** → **AWS service** → **EC2**.
2. Attach custom inline policy:

Copy Explain

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListObjectsOnly",
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket"
      ],
      "Resource": "arn:aws:s3:::my-demo-bucket"
    },
    {
      "Sid": "GetObjectsOnly",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::my-demo-bucket/*"
    }
  ]
}
```

3. Name the role `EC2ReadMyBucketRole`.

5.4 Launch an EC2 instance with the role

- EC2 Console → Launch instance → choose an Amazon Linux 2023 AMI → **Advanced details** → **IAM role:** `EC2ReadMyBucketRole`.
- Security group: allow outbound HTTPS (default).
- SSH in or use **Session Manager** (preferred—no inbound ports).
- Verify credentials:

```
curl -s http://169.254.169.254/latest/meta-data/iam/info
```

Copy Explain

Should show `EC2ReadMyBucketRole`.

5.5 Test S3 access from the instance

```
aws s3 ls                  # Fails with AccessDenied (no ListAllMyBuckets)
aws s3 ls s3://my-demo-bucket   # Succeeds, lists objects
```

Copy Explain

If you want the instance to perform `aws s3 ls` without a bucket name (i.e., list *all* buckets), add `s3:ListAllMyBuckets` on `*` to the role.

5.6 Verify least privilege with IAM Access Analyzer

- Console → **IAM** → **Access Analyzer** → **Create analyzer** (use **Organization** as the zone of trust).
- Wait for findings; you should see no external access to `my-demo-bucket`. Internal findings will list the EC2 role's allowed actions. ([Amazon Web Services][7], [Amazon Web Services][3])

5.7 Clean-up

- Terminate the EC2 instance.
- Detach/delete the role and the test user/group if no longer needed.

6. Modern extensions you should know

1. **IAM Identity Center** – central workforce SSO (supports Azure AD, Okta, etc.). Lets you map *permission sets* to AWS accounts so you never create IAM users again. ([AWS Documentation][1])
2. **Roles Anywhere** – your on-prem servers authenticate with X.509 certs and assume IAM roles, eliminating long-lived keys. ([AWS Documentation][8], [Amazon Web Services][2])
3. **Access Analyzer – Internal Findings (2025)** – automated reasoning across SCPs, identity & resource policies to spotlight unexpected org-internal access. ([Amazon Web Services][3])

4. **Verified Permissions** – offload fine-grained authZ for *your* SaaS/App to AWS so you stop writing custom permission code. ([Amazon Web Services][4])

7. Where to go next

- **IAM official docs – "Security best practices in IAM".** ([AWS Documentation][5])
- **AWS Security Blog's IAM tag** for weekly deep-dive posts and code samples. ([Amazon Web Services][9])
- **AWS re:Inforce 2025 identity track** recordings for architecture patterns. ([Amazon Web Services][10])
- **IAM Policy Simulator** (console) or `aws iam simulate-principal-policy` (CLI) to test policies before deployment.

TL;DR

Attach the **right role** with the **right policy** and your EC2 instance can indeed run:

```
aws s3 ls s3://your-bucket/
```

Copy Explain

—no hard-coded keys required. Start small, use Access Analyzer to double-check, and lean on identity-centered patterns to stay secure at scale.

AI TUTOR ACTIONS

Generate a practice quiz Summarize this unit Ask about this unit

Complete unit